

Project 1 — Bandit Algorithms on the Moon Game

CS5180 — Reinforcement Learning and Sequential Decision Making

In our first class, we played "Going to the Moon" — a guessing game where only yes/no feedback was returned. Without ever being told the rule, you discovered that good outcomes were worth repeating and bad ones worth avoiding. In this project, you formalize that experience. The Moon game is a multi-armed bandit (equivalently, a one-step MDP), and you will implement three classical algorithms for solving it — viewing each as a different instance of Generalized Policy Iteration.

Learning objectives

- Recognize the Moon game as a multi-armed bandit and explain what makes it a degenerate MDP.
- Frame ϵ -greedy, UCB, and gradient bandit as three improvement strategies sharing a common evaluation step.
- Implement, test, and compare the three algorithms with proper seed averaging.
- Interpret cumulative-reward and optimal-action curves to reason about exploration–exploitation tradeoffs.

What you will do

- Read the pre-filled ϵ -greedy agent. It is the template for the other two.
- Implement the UCB agent (TODOs in `act()`). Run its test cell — all three checks should print : "Correct".
- Implement the gradient bandit agent (TODOs in `act()` and `update()`). Run its test cell.
- Run the comparison harness. It executes each algorithm across 30 random seeds and produces mean $\pm$ std plots.
- Answer the five reflection questions in the notebook (a few sentences each).

Deliverables

Submit one .ipynb file containing:

- All three algorithm classes implemented, with their test cells passing.
- Rendered comparison plots (cumulative reward and fraction of optimal actions).
- Written answers to the five reflection questions.

Logistics

Download the starter notebook: [Lab Bandit Algorithms.ipynb](#)

Due: 6/22/2026

Submit via: CANVAS (Note: I only need a link to your Colab / Github where the your notebook is hosted)

Generative AI

This project sits in the Yellow tier of the course policy. AI tools may help you debug, explain math, or understand error messages. The algorithmic core — your `act()` and `update()` methods — must be yours. Add a one-line disclosure at the top of your notebook stating the tool and what you used it for.